

Lab 2 — Logstash

3.5 h · autonomous · graded deliverable

Session 4

Rules of the game

You work autonomously, alone or in pairs. Instructors clarify *missions*, never *solutions*. Allowed help: the **Course 2 slides** and the **official Logstash documentation** — nothing else.

`scripts/check.sh` is your referee: it prints PASS/FAIL per mission against the live cluster, and you can run it as often as you like. Everything you need was covered in Course 2 — when stuck, ask yourself *which slide was this?*, then find the matching documentation page.

Deliverable (graded /20, deadline: _____)

Your final `pipeline/pipeline.conf` + a 10-line write-up containing: (a) the mission 1 explanation, (b) grok vs. dissect — what you chose for the app logs and *why*, (c) your dead-letter strategy and how you verified that **nothing is silently lost**.

Starter kit & setup

```
lab2/
|-- docker-compose.yml      # ES + Kibana + Logstash (pipeline/ auto-reloads)
|-- generator/generate_logs.py # writes the three log sources into logs/
|-- pipeline/pipeline.conf  # inputs wired, filter{} empty: your canvas
|-- scripts/check.sh        # the referee
`-- LAB2.pdf                # this document
```

```
docker compose up -d
python3 generator/generate_logs.py      # keep it running in a terminal
docker compose logs -f logstash         # watch your rubydebug output
```

Three sources land in `logs/`, already mounted into the Logstash container at `/var/log/shopfast/`:

```
access.log : 81.2.69.142 - - [26/Jul/2026:01:47:03 +0000] "GET /checkout HTTP/1.1" 502
            1370 "-" "Mozilla/5.0..."
app.log    : 2026-07-26T01:47:11Z|payment|ERROR|order=48221|amount=59.90|msg=card
            declined
orders.csv : 48222,2026-07-26T01:48:02Z,FR,3,129.50
```

Classic trap

About 2% of the lines in *every* source are malformed. That is not a bug in the generator — it is the whole point of missions 6 and 7. Plan for it from the start.

🚩 Mission 1 — First flow, and a mystery

25 min

1. Bring the stack up, start the generator, and confirm raw events from **all three sources** appear in the Logstash output.
2. Now run `docker compose restart logstash`. Observe what is re-read and what is not. Then `docker compose down && docker compose up -d`. Observe again.
3. **Write-up:** explain both observations with the correct Logstash concept, and name the setting you would use to force a full replay during development.

💡 Expected result

Events from the three files visible in `docker compose logs -f logstash`; a correct explanation of what survives a restart, what survives a down, and why.

🚩 Mission 2 — Tame the nginx source

45 min

For events coming from `access.log` (tag `web`), produce documents where:

- the line is fully parsed — no giant `message`-only events;
- `status` and `bytes` are **integers**, named exactly like that;
- `@timestamp` is the time **inside the log line**, not the ingestion time;
- the raw parsed timestamp string no longer pollutes the document.

💡 Expected result

Typed fields and a correct `@timestamp` in `rubydebug`. The two `M2` lines of `check.sh` turn green once mission 6 ships to Elasticsearch.

📖 Doc pointers

Logstash reference: *grok filter*, *grok patterns list*, *date filter*, *mutate filter*.

🚩 Mission 3 — Tame the app source

35 min

For events from `app.log` (tag `app`), produce documents with:

- fields `service`, `level`, `order` (integer), `amount` (float), `msg`;
- `@timestamp` taken from the line;
- parsing implemented with **grok or dissect — your choice**.

Write-up: justify the choice in 3 lines (think: format regularity, failure behavior, performance). A “both would work, here is why I picked X” answer is a good answer.

🚩 Mission 4 — Tame the CSV source

25 min

For events from `orders.csv` (tag `orders`):

- columns are `order_id`, `ts`, `customer_country`, `items`, `amount`;
- `items` is an integer, `amount` a **float**;
- `@timestamp` comes from the `ts` column.

💡 Expected result

Typed order documents in `rubydebug`; the *M4* line turns green later.

📄 Doc pointers

Logstash reference: *csv filter*.

🚩 Mission 5 — Enrich and clean

30 min

1. Enrich web events with **geolocation** derived from the client IP (a `geoip` object with at least country and coordinates).
2. The `/healthz` probes are internal monitoring noise: they must **not exist** in your output. Anywhere.
3. Bonus polish: parse the browser/OS out of the user-agent string.

💡 Expected result

Web events carry `geoip.country_name` and coordinates; zero `/healthz` documents. `check.sh` *M5* lines green later.

🚩 Mission 6 — Ship it. Lose nothing.

40 min

Replace/extend the output section so that:

- each source goes to its own dated index: `shopfast-web-YYYY.MM.dd`, `shopfast-app-YYYY.MM.dd`, `shopfast-orders-YYYY.MM.dd`;
- **any** event that failed parsing — from any source — lands in `shopfast-deadletter`;
- no event path leads nowhere: **nothing may be silently lost**.

```
./scripts/check.sh          # the goal: every line PASS
```

⚠️ Classic trap

Date-pattern index names use the *event's* `@timestamp`. If missions 2–4 got the date wrong, your indices will be named after the wrong day — check `GET _cat/indices?v` with Course 1 eyes.

🚩 Mission 7 — Prove it

15 min

1. Let the generator run a few minutes, then run `check.sh`: **all PASS**.
2. In Dev Tools, write one query that counts dead-letter events *per source file* — paste it and its result into your write-up.
3. Skim 3 dead-letter documents: can you tell *why* each one failed? One line each.

💡 Why this matters

In the final project, dead-letter handling and “nothing silently lost” are worth real points on an *unseen* dataset — this mission is your rehearsal.

Bonus — One pipeline per source (+2)

Split your monolithic `pipeline.conf` into three isolated pipelines using `pipelines.yml`, without changing any behavior (`check.sh` must still be all green).

Hint-free zone. The documentation page you need is about multiple pipelines — and remember how the Docker image decides what to run.

Ship `pipeline.conf` + write-up before the deadline.

Next session: Course 3 — Kibana, where your clean indices finally become pictures.